Task Description:

Task-1: Count the number of primitive operations executed below and determine the best & the worst cases: (1 points)

Algorithm: *arrayMin*(A, n)

	cost	* times	= total
currentMin ← A[0]	2	1	2
i ← 1	1	1	1
while *i* ≤ *n* − 1 **do**	2	n	2n
if *currentMin* ≥ A[*i*] **then**	2	n-1	2(n-1)
currentMin ← A[*i*]	2	0~n-1	0~2(n-1)
i ← *i* + 1	2	n-1	2(n-1)
return *currentMin*	1	1	1

Best: do NOT let 'currentMin ← A[i]' work but 'while' still goes ⇒ O(n) *

Worst: executed 'currentMin ← A[i]' as many as possible ⇒ O(n) *

space: currentMin, i ⇒ O(1) *

Task-2: Determine the Big-O notation for: (3 points)

a) $2 + n(2 + 3n)$

$2 + 2n + 3n^2$ ⇒ $O(n^2)$ *

b) $n + 2(n + 3n)\, n + \frac{n}{2}$

$n + (2n + 6n)n + \frac{n}{2} = n + 2n^2 + 6n^2 + \frac{n}{2} = 8n^2 + n + \frac{n}{2}$ ⇒ $O(n^2)$ *

c) $n^3 \log n + 2n + 1 + 3n^2 + n(\log n)^2$

↳ $n^3 \log n$ ⇒ $O(n^3 \log n)$ *

Task-3: Determine the Complexity Of The Following Small Functions: (6 points)

a)
```
for (i = sum = 0; i < n; i++)
    sum += a[i];
```

run "n" times ⇒ O(n) *

space: sum, i ⇒ O(1) *

b)
```
for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
a[i][j] = i*j;
```

n * n = n² ⇒ O(n²) *

space: i, j ⇒ O(1) *

c)
```
for (i = n; i >= 1; i--)
    for (j = i; j <= n; j++)  /* Note that the value of the inner loop variable (j)  */
        ...                   /* depends on the value of the outer loop variable (i) */
```

if n=3, ① i=3 : j=3, 3≤3, run
j=4, 4≤3, ×

② i=2 : j=2, 2≤3, run
j=3, 3≤3, run
j=4, 4≤3 ×

d)
```
for (i = 1; i <= n; i++)
    for (j = i; j <= i; j++)      /* Note that the value of the inner loop variable (j)  */
        ...                       /* depends on the value of the outer loop variable (i) */
```

③ $i=1$: $j=1$, $1\le 3$, run
$j=2$, $2\le 3$, run
$j=3$, $3\le 3$, run
$j=4$, $4\not\le 3$ ×

⇒ j start with i, and i depends on n

inner : 1, 2, 3, ..., n

total : $1+2+3+\dots+n = (n+1)*n/2 \Rightarrow O(n^2)$ ※

Space : i, j ⇒ O(1) ※

if n=3, ① $i=1$: $j=1$, $1\le 1$, run
$j=2$, $2\not\le 1$ ×

② $i=2$: $j=2$, $2\le 2$, run
$j=3$, $3\le 2$ ×

③ $i=3$: $j=3$, $3\le 3$, run
$j=4$, $4\not\le 3$ ×

e)
```
for (i = 0; i < n; i++)
    for (j = n; j > 1; j/=2)
        ...
```

⇒ total : 1+1+1+...+1 for n times ⇒ O(n) ※

Space : i, j ⇒ O(1) ※

if n=16, inner $j=16$: $j=16$, $16>1$, run → $j=8$
$j=8$, $8>1$, run → $j=4$
$j=4$, $4>1$, run → $j=2$
$j=2$, $2>1$, run → $j=1$
$j=1$, $1\not>1$ ×

outer from 0 to 15 ⇒ n times

⇒ total : $n*\log n \Rightarrow O(n\log n)$ ※

Space : i, j ⇒ O(1) ※

⇒ 4 times, so $\log_2 16 = 4$

f)
```
int factorial (int n)
{
    if (n <= 1)
      return 1;
    else
      return n * factorial(n-1);
}
```

if n=4, $4\not\le 1$, $4*f(4-1)=4*f(3)$ ↑
$3\not\le 1$, $3*f(3-1)=3*f(2)$ ↑
$2\not\le 1$, $2*f(2-1)=2*f(1)$ ↑
$1\le 1$, return 1

⇒ so n=4 execute things 4 times (n)

⇒ total : n * constant ⇒ O(n) ※

space recursion keeps n calls

⇒ O(n) ※